

M18 - Project Handbook

Portfolio evidence system | RC2

1. What changes in M18

Until now, lessons told you what to learn and labs told you where to work. In M18 the business problem comes first. You must decide grain, keys, architecture, checks, rerun behavior and evidence. That shift is intentional: interviews and junior engineering work rarely arrive as numbered instructions.

2. The universal project cycle

- Read the brief and list business questions before coding.
- Profile every source and write down grain, business keys, null/duplicate risks and timestamps.
- Draw a small architecture before creating folders.
- Preserve raw source evidence and record batch/source metadata.
- Build the smallest correct ingestion path first.
- Add validation, quarantine and reconciliation before trusted publish.
- Add modeling/transformation with explicit output grain.
- Make the workflow repeatable with commands or orchestration.
- Rerun from a clean state and prove idempotency.
- Trigger the supplied failure; save the failing evidence before repair.
- Repair, rerun and save the new evidence.
- Finish README, architecture, runbook, limitations and defense only after the system works.

3. Repository structure that employers can understand

- README.md - problem, architecture, setup, run order, evidence and limitations.
- src/ or pipeline code - ingestion/transformation logic.
- data/ - only safe sample data; never real employer/customer data.
- sql/, dbt/, dags/, spark/ as required by the project.
- tests/ - automated checks and business assertions.
- evidence/ - logs, checker output, reconciliations and screenshots where useful.
- runbook.md - rerun, backfill, recovery and rollback.
- architecture.md - source -> landing/raw -> trusted -> serving flow and key decisions.
- failure_pack/ or incidents/ - supplied failures and your repair evidence.

4. README standard

- State the business problem in plain English.
- List the stack, but explain why each tool exists.
- Show the architecture flow and trusted output grain.
- Give exact setup and run commands in the correct order.
- Describe quality rules and what happens to bad data.
- Explain rerun/idempotency and backfill behavior.
- List failure scenarios you tested and what recovered them.

- Show key outputs or metrics without pretending screenshots are tests.
- State limitations and the next production improvements.

5. Architecture decisions

A good diagram is not a cloud-logo poster. Every box must have a responsibility. For each boundary explain what enters, what leaves, what is persisted, what is trusted, and what happens on failure. Prefer the simplest architecture that satisfies scale, reliability and security requirements.

6. Data quality and reconciliation

- Required-key checks protect identity and grain.
- Type/date/range checks protect transformation correctness.
- Duplicate checks distinguish exact duplicates from legitimate updates.
- Relationship checks catch orphan references.
- Reconciliation compares source counts/totals with accepted + quarantined + deduplicated outcomes.
- A quality failure must block trusted publish when the rule is critical.

7. Rerun and idempotency proof

Do not write "idempotent" only in the README. Run the same batch twice and prove trusted logical row counts and totals do not double. Save both runs. For incremental designs, record the business key, watermark, merge rule and late-data policy.

8. Failure-evidence standard

- Save the first failing log/checker output.
- Name the root cause, not only the symptom.
- Record the smallest safe repair.
- Rerun the failed path and then the full safety checks.
- Explain why the repair would still work after another rerun or at larger scale.

9. Security and configuration

Use sample `.env.example` or configuration templates, never real keys. Keep credentials outside Git. Treat PII as a data-design concern: identify fields, restrict access, mask when appropriate and document who needs raw access.

10. Project defense

You should be able to explain the output grain, one rejected design option, one quality rule, one rerun/backfill decision, one failure you debugged, one security choice, one performance/cost trade-off and what changes at 10x volume without reading a script.

11. GitHub publication checklist

- Repository opens with a useful README.

- No secrets, local virtual environments, large raw extracts or temporary build files committed.
- Meaningful commit history exists.
- Setup works from a clean clone or the limitation is stated honestly.
- Architecture/runbook/tests are visible.
- Project title and description describe the business outcome, not only the tools.

12. Independence curve

P1 still gives a clear starter repository. P2 removes many implementation hints. P3 expects you to make distributed-storage/performance decisions. The capstone gives only requirements, source definitions, constraints, deliverables and a later incident pack. This is deliberate preparation for interviews and real work.

13. Completion rule

A project passes only after checker/test evidence and human review of architecture, failure recovery, README and defense. A green checker cannot compensate for leaked secrets, unsafe reruns, silently bad data or an explanation you cannot defend.